

# Rank-GRPO

Training LLM-Based Conversational Recommender Systems  
with Reinforcement Learning

Zhu et al., ICLR 2026 · UVA / Netflix / Cornell

Tomás Ariztía - Rodrigo Meza - Vicente Muñoz

DCG@k:N

# De predecir a conversar

- RS tradicionales: predicen en pasivo (historial, implícito)
- CRS: agentes proactivos, permite establecer preferencias en lenguaje natural
- El LLM es el centro: entiende y genera la recomendación
- Ítems en lenguaje natural > IDs numéricos

## RS tradicional

Entrada: historial · feedback implícito → predicción pasiva

## CRS

Entrada: diálogo explícito → recomendación generada

lenguaje natural

# 3 fallas de los LLMs pre-entrenados

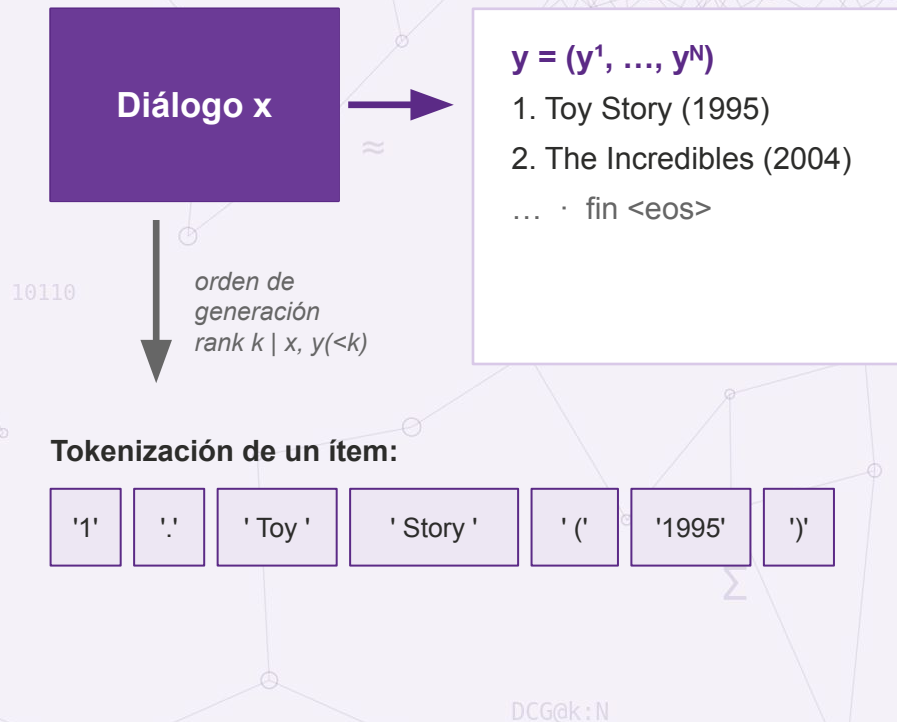
1. Ítems fuera de catálogo: OOC (alucinaciones)
2. Formato de output incorrecto
3. La calidad cae al final de la lista

“Recomiéndame thrillers de ciencia ficción”

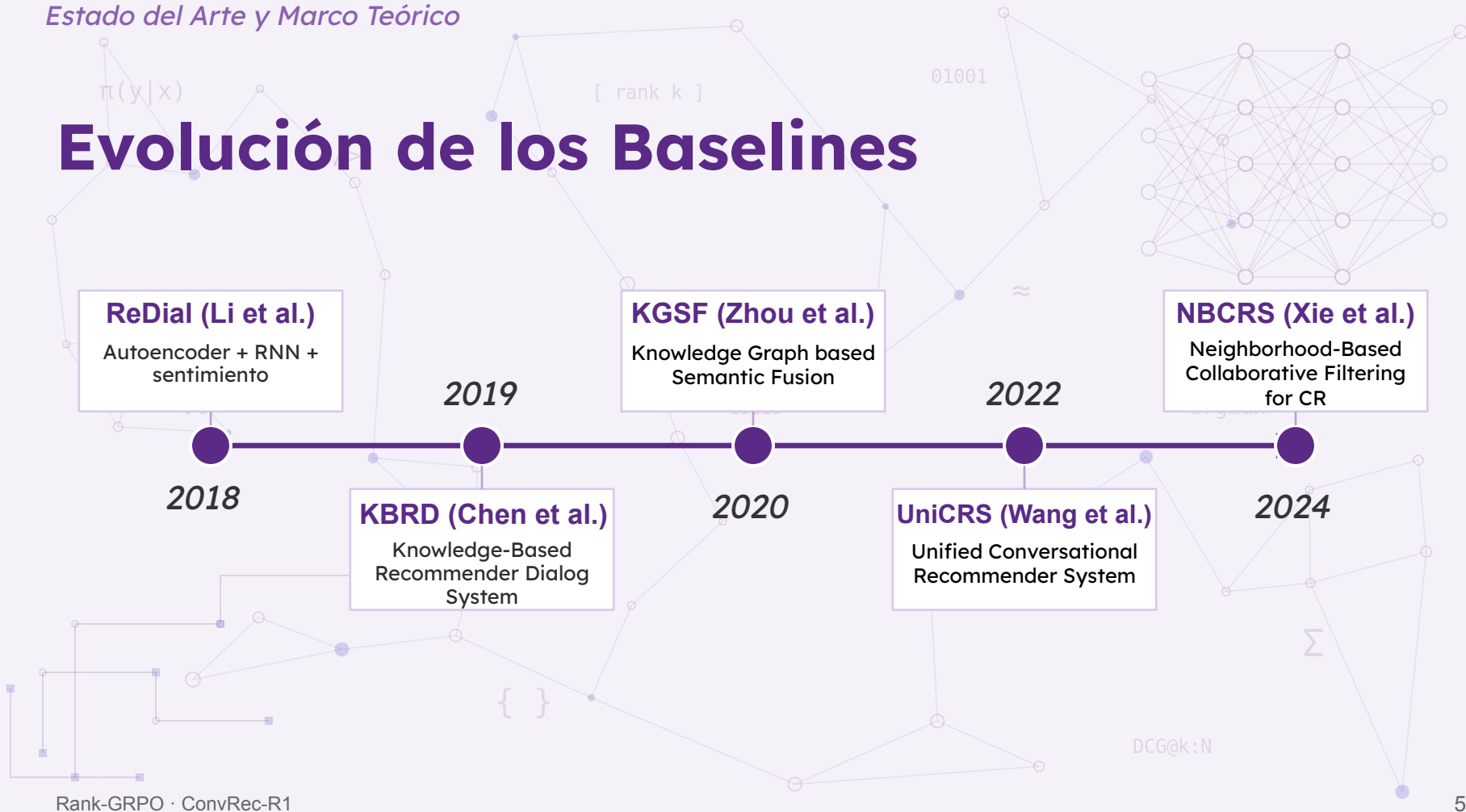
1. Inception (2010)
2. {title: "Arrival"} -> Formato roto
3. The Matrix (1999)
4. Inception 2 (2012) -> No existe
- ...
20. The Notebook (2004) -> La calidad decae

# Problema de Recomendación

- Problema de decisión secuencial
- Política  $\pi_{\theta}(y|x)$  genera lista rankeada de N ítems
- Cada ítem es texto: “The Incredibles (2004)”, no un ID
- Generación posición por posición condicionada a ‘x’ y a previas recomendaciones
- Reward: Recall@k y nDCG@k sobre el ground truth



# Evolución de los Baselines



# Métodos basados en prompting de LLMs

- GPT-4o / GPT-4o-mini zero-shot: prompt directo
- Problema: se salen del catálogo
- CRAG: GPT-4o multi-paso

## Zero-shot (GPT-4o / -mini)

Prompt directo · 1 llamada · se sale del catálogo

## CRAG (GPT-4o multi-paso)

Extraer → reflexionar → reordenar

**5-7** llamadas a la API por recomendación

# Aprendizaje Reforzado para LLMs

*Objetivo: ajustar  $\theta$  para mayor DCG esperado. El reward solo llega al final de la lista, no por token.*

| Concepto RL     | En el LLM recomendador                      |
|-----------------|---------------------------------------------|
| <b>Estado</b>   | La conversación $x$ + lo ya generado        |
| <b>Acción</b>   | Generar el siguiente token                  |
| <b>Política</b> | El LLM mismo (parámetros $\theta$ )         |
| <b>Reward</b>   | DCG@N de la lista completa vs. ground truth |

# Group Relative Policy Optimization (GRPO)

- GRPO (Shao et al., 2024): generan G respuestas y se estiman ventajas usando recompensas relativas
- Se debe maximizar la siguiente función para ajustar los parámetros  $\theta$

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{X \sim \mathcal{D}_{\text{RL}}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | x)} \left[ \frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min(w_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(w_{i,t}(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_{i,t}) \right]$$



# Problema con GRPO

- Problema: reward uniforme a cada token
- Asignación no causal: el final cobra crédito del inicio



# Los 2 problemas de GRPO en ranking

- 1) Crédito no causal: mismo reward a todos los tokens
- → buenas notas a recomendaciones malas del final
- 2) Desajuste de granularidad: ratio/token vs. reward/secuencia
- GSPO ajusta a secuencia, pero sigue sin servir para ranking

## Problema 1 · crédito no causal

rank 1

rank 2

rank ...

rank 20

*mismo reward → todos los tokens*

## Problema 2 · granularidad



Token



Secuencia (GSPO)



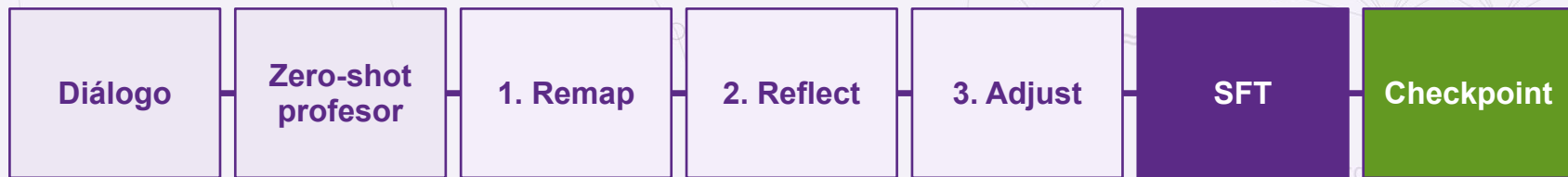
Rango ✓

# Aporte: ConvRec-R1

3 contribuciones

| Problema                                         | Contribución                                                                |
|--------------------------------------------------|-----------------------------------------------------------------------------|
| No hay datos de ranking anclados al catálogo     | <b>(Remap-Reflect-Adjust) → genera D_SFT y se usa para SFT</b>              |
| Credit assignment desalineado en RL (GRPO/GSPO)  | <b>Rank-GRPO: el rango es la acción · reward masking · media geométrica</b> |
| LLMs grandes muy costosos en producción (GPT-4o) | <b>0.5B-3B igualan o superan a GPT-4o, a una fracción del costo</b>         |

# Etapa 1: SFT con Remap-Reflect-Adjust



El “teacher” (GPT-4o) genera un dataset enriquecido (D\_SFT):

Remap ancla cada recomendación al catálogo por similitud semántica → Reflect re-puntúa por relevancia contextual → Adjust corrige el sesgo de popularidad. El output D\_SFT entrena al estudiante (par conversación, lista de 20).

- Modelo ‘checkpoint’ pasa a etapa 2.

# Etapa 1: Ejemplo flujo

| Etapa              | Qué hace                                                      | Ejemplo                                                                                             |
|--------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Zero-shot profesor | GPT-4o: 20 recomendaciones <b>sin</b> restricción de catálogo | - 3° Mutiny on the Bounty (1935)<br>- 7° The Black Swan (1942)<br>- ...                             |
| 1. Remap           | Ancla al catálogo por embeddings (mediante Stella 400M)       | - Mutiny on the Bounty (1935) → The Bounty (1984)<br>- The Black Swan → Master and Commander (2003) |
| 2. Reflect         | GPT-4o juez puntúa por relevancia; se suma a Remap            | The Bounty 3°→5°; Master and Commander 7°→6°                                                        |
| 3. Adjust          | Multiplicador $w$ y sesgo $b$ corrigen popularidad.           | Master and Commander 6°→4°                                                                          |
| SFT                | Fine-tuning del estudiante con D_SFT (conversación, lista 20) | Resultado = modelo checkpoint para la Etapa 2                                                       |

# Remap

Anclar las recomendaciones del profesor al catálogo objetivo mediante similitud semántica de metadatos.

$$s_{\text{remap}} = p \cdot (S_{\text{item-item}} + I_{\text{ic}}) + \lambda \cdot S_{\text{conv-item}}$$

**P:** “peso” según su posición ( $1/\sqrt{k}$ )

**S<sub>item-item</sub>:** “similitud semántica” Matriz coseno entre embeddings de metadata. Mapea cada ítem de  $C\Theta$  al catálogo C. Ej: si el profesor dijo “Inception 2” (no existe), el embedding lo acerca a “Inception (2010)”.

**i<sub>ic</sub>:** “indicador in-catalog” 1 si el ítem del profesor ya existe idénticamente en C.

**S<sub>conv-item</sub>:** “similitud conversación”. Coseno entre el embedding del diálogo x y cada ítem del catálogo. Captura relevancia contextual.

**λ:** Hiperparametro de peso entregado al contexto (1 en el paper).

# Reflect

Refinar la calidad del ranking usando al profesor como evaluador cualitativo de relevancia contextual.

$$\mathbf{S}_{\text{reflect}} = \mathbf{S}_{\text{remap}} + \gamma \cdot \mathbf{r}_{\text{reflect}}$$

**$\gamma$ :** hiperparámetro de escala que controla cuánto peso tiene la opinión del juez (0.5 en el paper).

**$\mathbf{r}_{\text{reflect}}$ :** vector sparse que contiene las calificaciones numéricas que el LLM profesor asigna a los ítems candidatos actuando como "juez"

## ¿COMO FUNCIONA?

1. Se toman los top- $N_r > N$  candidatos del Remap (ej: top 100)
2. GPT-4o evalúa cada candidato en escala de L niveles ( $L=5$  en paper):  
-L (malo) ... 0 (normal) ... +L (bueno)
3. Las calificaciones forman  $\mathbf{r}_{\text{reflect}}$ , se escalan con  $\gamma$

# Adjust

Corregir sesgos de popularidad del profesor alineando puntuaciones con la distribución real del ground truth.

$$\mathbf{s}_{\text{final}} = \mathbf{w} \odot \mathbf{s}_{\text{reflect}} + \mathbf{b} \approx$$

**w**: sesgo multiplicativo por ítem

**b**: sesgo aditivo por ítem

$\odot$ : producto Hadamard (elemento a elemento)

## ¿COMO SE CALCULAN?

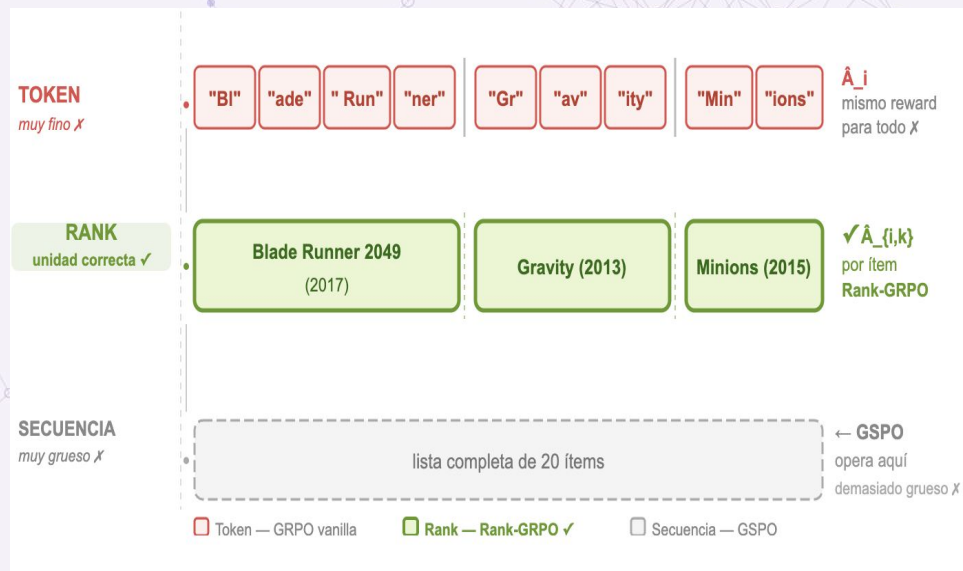
Se optimizan maximizando la log-verosimilitud multinomial de observar los ítems groundtruth del dataset de entrenamiento.



# Etapa 2: Rank-GRPO

## El rango como unidad de acción

- LLM post-SFT ya conoce el catálogo; falta rankear bien
- GRPO: un  $\hat{A}_i$  para toda la secuencia  $\rightarrow$  mismo crédito
- Rank-GRPO:  $\hat{A}_{\{i,k\}}$  por rango  $\rightarrow$  cada ítem responde por sí
- Doble suma sobre respuestas  $i$  y rangos  $k$  (novedad)



# Objetivo Rank-GRPO + Advantage

$$\mathcal{J}_{\text{Rank-GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}_{\text{RL}}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[ \frac{1}{GN} \sum_{i=1}^G \sum_{k=1}^N \min(w_{i,k}(\theta) \hat{A}_{i,k}, \text{clip}(w_{i,k}(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_{i,k}) \right]$$

- Advantage Normalizada:

$$\hat{A}_{i,k} = \frac{r(x, y_i^{(k)}) - \text{mean}_{i'} [r(x, y_{i'}^{(k)})]}{\text{std}_{i'} [r(x, y_{i'}^{(k)})]}$$

**G** = listas por conversación (grupo)

**N** = 20 ítems por lista

**w**<sub>{i,k}</sub> = importance ratio del ítem

**Â**<sub>{i,k}</sub> = ventaja del rank k vs. grupo

**clip** [1-ε, 1+ε] = controla updates  
(ε=0.06-0.08)

# Importance ratio estabilizado con media geométrica

$$w_{i,k}(\theta) = \frac{\bar{\pi}_{\theta}(y_i^{(k)}|x)}{\bar{\pi}_{\theta_{\text{old}}}(y_i^{(k)}|x)}$$

$$\bar{\pi}_{\theta}(y_i^{(k)}|x) = \left( \prod_{t=1}^{|y_i^{(k)}|} \pi_{\theta}(y_{i,k,t}|x, y_{i,k,<t}) \right)^{1/|y_i^{(k)}|} = \exp \left( \frac{1}{|y_i^{(k)}|} \sum_{t=1}^{|y_i^{(k)}|} \log \pi_{\theta}(y_{i,k,t}|x, y_{i,k,<t}) \right).$$

# Reward shaping + Penalizaciones



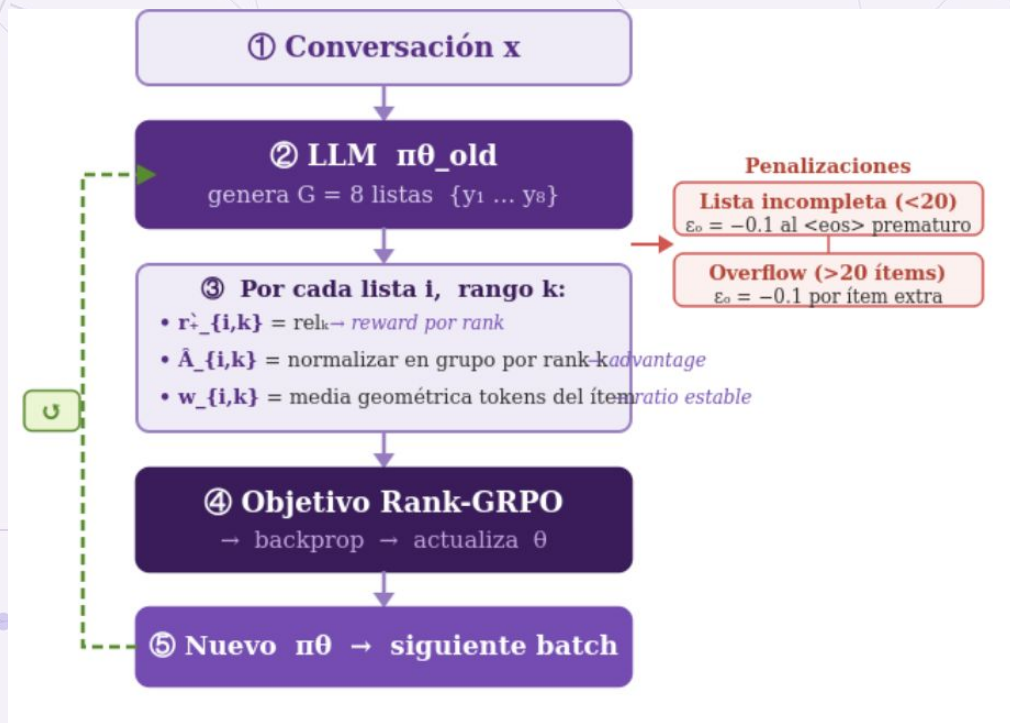
$$r(x, y_i^{(k)}) \triangleq \text{DCG@}\mathbf{k:N} = \sum_{j=\mathbf{k}}^N \frac{\text{rel}_j}{\log_2(j+1)}.$$

$$r_{\text{exp}_\Gamma}(x, y_i^{(k)}) \triangleq \sum_{j=k}^N \frac{\text{rel}_j}{\Gamma(j-k)} = \text{rel}_k + \frac{1}{\Gamma} \cdot r_{\text{exp}_\Gamma}(x, y_i^{(k+1)}).$$

- Rank  $k$  recibe crédito solo desde la posición  $k$  hasta  $N$
- $\text{exp}_\infty$ :  $r_{\{i,k\}} = \text{rel}_k \rightarrow$  solo relevancia inmediata

DCG@k:N

# Ciclo de entrenamiento RL



# Setup experimental

- Dataset: Reddit-V2 mayor benchmark CRS (~400k conv.)
- Backbones: Qwen2.5-0.5B · Llama-3.2-1B · Llama-3.2-3B
- Modelos  $\leq 3B$ : restricciones de latencia industrial
- Métricas: Recall@k y NDCG@k ( $k \in \{5, 10, 15, 20\}$ )

## Dataset · Reddit-V2

Train 383.013 · Val 9.421 · Test 10.972

Test tomado 1 mes después (generalización temporal)

## Backbones · $\leq 3B$

Qwen2.5-0.5B · Llama-3.2-1B · Llama-3.2-3B

Recall@k = ítems del GT en los primeros k

NDCG@k: rank 1 pesa 1; rank 20  $\approx 0.23$

# Resultados principales vs Mejores Baselines

| Método                                                                | R@20          | N@20          |
|-----------------------------------------------------------------------|---------------|---------------|
| GPT-4o zero-shot                                                      | 0.2147        | 0.1197        |
| CRAG (GPT-4o, 5–7 API calls)                                          | 0.2212        | 0.1227        |
| Qwen-0.5B + SFT + Rank-GRPO( $\exp^\infty$ )                          | 0.2047        | 0.1079        |
| Llama-1B + SFT + Rank-GRPO( $\exp^\infty$ )                           | 0.2165        | 0.1153        |
| <b>Llama-3B + SFT + Rank-GRPO(<math>\exp^\infty</math>) ← ganador</b> | <b>0.2368</b> | <b>0.1283</b> |

- Qwen-0.5B + Rank-GRPO( $\exp^\infty$ ) supera a GPT-4o-mini
- Llama-1B + Rank-GRPO( $\exp^\infty$ ) iguala a GPT-4o zero-shot
- Llama-3B + Rank-GRPO( $\exp^\infty$ ) supera a CRAG
- Ambas etapas suman: SFT catálogo, RL calidad posicional

Resultados off-policy ( $\mu=2$ )

# Training dynamics

- Rank-GRPO( $\exp^\infty$ ): ranks 1,5,10,15,20 suben juntos
- GRPO vanilla: rangos finales (15,20) se quedan atrás
- Rank-GRPO converge más rápido y a mayor NDCG
- On-policy vs off-policy: off-policy reutiliza, más varianza

INSERTAR

## Figura 2 / 3 — Training dynamics (Llama-3B)

Anotar: “ranks 15-20 suben” sobre Rank-GRPO; “quedan atrás” sobre GRPO vanilla. Panel NDCG (derecha): marcar ventaja de Rank-GRPO( $\exp^\infty$ ).

DCG@k:N



# Análisis cualitativo

- Zero-shot (NDCG=0.000): repite películas ya mencionadas
- SFT (NDCG=0.249): lista anclada al catálogo, formato OK
- RL (NDCG=0.444): ítems relevantes en posiciones altas
- El mismo 0.5B pasa de NDCG=0 a NDCG=0.44

| Zero-shot · NDCG 0.000                                          | SFT · NDCG 0.249                                      | RL · NDCG 0.444                                                      |
|-----------------------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------|
| Avatar (ya visto)<br>Avengers (ya visto)<br>Godzilla (ya visto) | Mad Max #2 ✓<br>Gravity #5 ✓<br>Ready Player One #8 ✓ | Mad Max #1 ✓<br>Blade Runner 2049 #4 ✓<br>Gravity #5 · R.P. One #7 ✓ |

## Ground truth

Mad Max: Fury Road  
Gravity  
Blade Runner 2049  
Ready Player One  
Life of Pi

# Conclusiones

- Stage 1 permite RLVR: rewards verificables → alineación sin anotadores
- Rango como unidad resuelve el crédito no-causal
- Mejora calidad consistente en toda la lista, especialmente al final
- LLMs de 0.5B–3B igualan o superan a GPT-4o en CRS }

Remap–Reflect–Adjust → SFT

Rank-GRPO → RL

LLM open-source ≈ GPT-4o

# Referencias

- Zhu, Y., Steck, H., Liang, D., He, Y., Ostuni, V., Li, J. & Kallus, N. (2026). Rank-GRPO: Training LLM-based Conversational Recommender Systems with Reinforcement Learning. ICLR 2026.
- Shao, Z. et al. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models (GRPO). arXiv:2402.03300.
- Zheng, C. et al. (2025). Group Sequence Policy Optimization (GSPO). arXiv:2507.18071.
- He, Z. et al. (2023). Large Language Models as Zero-Shot Conversational Recommenders. CIKM 2023. [dataset Reddit-v2]
- Zhu, Y. et al. (2025). Collaborative Retrieval for LLM-based Conversational Recommender Systems (CRAG). ACM Web Conference (WWW) 2025.
- Li, R. et al. (2018). Towards Deep Conversational Recommendations (ReDial). NeurIPS 2018.
- Chen, Q. et al. (2019). Towards Knowledge-Based Recommender Dialog System (KBRD). EMNLP 2019.
- Zhou, K. et al. (2020). Improving Conversational Recommender Systems via Knowledge Graph based Semantic Fusion (KGSF). KDD 2020.
- Wang, X. et al. (2022). Towards Unified Conversational Recommender Systems via Knowledge-Enhanced Prompt Learning (UniCRS). KDD 2022.
- Xie, Z. et al. (2024). Neighborhood-based Collaborative Filtering for Conversational Recommendation (NBCRS). RecSys 2024.

# Anexo - Prompts de generación de datos

## Prompt to Generate Metadata for Items Known to the Teacher LLM $\Theta$

Summarize the metadata for the movie movie\_name (release year) based on the information in the format of:

**Movie\_Name (Release\_Year)**

**Keywords:**

<comma-separated keywords>

**Plots:**

<one-paragraph plot summary>

You should separate the keywords with commas without numbering.  
The plot should be one paragraph.

## Prompt to Generate Metadata for Items Unknown to the Teacher LLM $\Theta$ with RAG and ICL

Summarize the metadata for the movie movie\_name (release year) based on the information in the format of:

**Movie\_Name (Release\_Year)**

**Keywords:**

<comma-separated keywords>

**Plots:**

<one-paragraph plot summary>

You should separate the keywords with commas without numbering.  
The plot should be one paragraph.

Here are the information related to the target movie: {movie\_info}

Here are some examples for the generated metadata for different movies: {examples}

# Anexo - Parámetros y Variables GRPO

- $x$ : conversación de entrada (contexto), tomada del dataset  $D_{RL}$
- $D_{RL}$ : datos de entrenamiento RL (conversaciones + ground truth)
- $E[\cdot]$ : esperanza sobre las conversaciones y los grupos de respuestas
- $G$ : n° de respuestas (listas) que se generan por cada  $x$
- $y_i$ :  $i$ -ésima respuesta generada
- $|y_i|$ : n° de tokens de la respuesta  $y_i$
- $y_{i,t}$ :  $t$ -ésimo token de la respuesta  $i$
- $\theta$ : parámetros del LLM que se entrena (la política actual  $\pi_\theta$ )
- $\theta_{old}$ : parámetros de la política anterior, que generó las muestras (off-policy)
- $\pi_\theta(\cdot|x)$  política actual
- $\pi_{\theta old}(\cdot|x)$  política de muestreo
- $w_{i,t}(\theta)$ : ratio de importancia =  $\pi_\theta(y_{i,t} | x, y_{i,<t}) / \pi_{\theta old}(y_{i,t} | x, y_{i,<t})$ , corrige el desfase entre la política nueva y la que muestreó
- $\hat{A}_{i,t}$ : ventaja (igual para todos los tokens de  $y_i$ ): qué tan buena fue esa respuesta respecto al grupo  $\rightarrow \hat{A}_i = (r_i - \text{media}) / \text{desviación estándar}$
- $r(x, y_i)$ : recompensa de la respuesta  $i$
- $\text{clip}(w, 1-\epsilon, 1+\epsilon)$ : limita el ratio para que la política no se aleje demasiado
- $\epsilon$ : hiperparámetro de recorte (cuánto puede cambiar la política por paso)
- $\min(\cdot, \cdot)$  toma la versión recortada cuando el cambio sería excesivo
- $(1/G) \sum_i$  promedia sobre las  $G$  respuestas del grupo
- $(1/|y_i|) \sum_t$  promedia sobre los tokens de cada respuesta

argmax

$\Sigma$

DCG@k:N

# Anexo - Prompt LLM-as-a-Judge

## Prompt for LLM-as-a-Judge in the *reflect* step

Pretend you are a movie recommender system. I will give you a conversation between a user and you (a recommender system) as well as some movie candidates from our movie database.

You need to rate each retrieved movie as recommendations into five levels based on the conversation:

2 (great), 1 (good), 0 (normal), -1 (not good), -2 (bad).

Here is the conversation: {context}

Here are the movie candidates: {rec\_titles}.

You need to reply with the rating of each movie in a line, in the form of movie\_name (release\_year)###rating

DCG@k:N

# Anexo - Análisis de Gradiente GRPO

$$\begin{aligned}
 \nabla_{\theta} \mathcal{J}_{\text{GRPO}}(\theta) &= \nabla_{\theta} \mathbb{E}_{\pi_{\theta_{\text{old}}}} \left[ \frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} w_{i,t}(\theta) \hat{A}_{i,t} \right] = \mathbb{E}_{\pi_{\theta_{\text{old}}}} \left[ \frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \hat{A}_{i,t} \nabla_{\theta} w_{i,t}(\theta) \right] \\
 &= \mathbb{E}_{\pi_{\theta_{\text{old}}}} \left[ \frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \hat{A}_{i,t} w_{i,t}(\theta) \nabla_{\theta} \log w_{i,t}(\theta) \right] \quad (\text{Log-derivative trick}) \\
 &= \mathbb{E}_{\pi_{\theta_{\text{old}}}} \left[ \frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} w_{i,t}(\theta) \hat{A}_{i,t} \nabla_{\theta} \log \pi_{\theta}(y_{i,t}|x, y_{i,<t}) \right] \\
 &\propto \mathbb{E}_{\pi_{\theta_{\text{old}}}} \left[ \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \underbrace{w_{i,t}(\theta)}_{\text{token (t)-level importance ratio}} \cdot \underbrace{\hat{A}_i}_{\text{sequence (i)-level advantage}} \cdot \underbrace{\nabla_{\theta} \log \pi_{\theta}(y_{i,t}|x, y_{i,<t})}_{\text{token (t)-level gradient of log-likelihood}} \right].
 \end{aligned}$$

DCG@k:N



# Anexo - Prompt para generar recomendaciones

## Prompt to Generate Recommendations

Pretend you are a movie recommender system.  
I will give you a conversation between a user and you (a recommender system).  
Based on the conversation, you need to reply with 20 recommendations.

List the standardized English title of each movie in each line in the form of  
"movie name" (release\_year) with NO extra words or sentences.

Here is the conversation: {context}



# Anexo - comparación ConvRec1 y baselines en off-policy

| Method                                                                                                                  | R@5           | R@10          | R@15          | R@20          | N@5           | N@10          | N@15          | N@20          |
|-------------------------------------------------------------------------------------------------------------------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
| <b>Traditional and Transformer-based CRS</b>                                                                            |               |               |               |               |               |               |               |               |
| Redial (Li et al., 2018)                                                                                                | 0.0103        | 0.0228        | 0.0274        | 0.0322        | 0.0073        | 0.0113        | 0.0128        | 0.0143        |
| KBRD (Chen et al., 2019)                                                                                                | 0.0444        | 0.0813        | 0.1058        | 0.1223        | 0.0305        | 0.0418        | 0.0490        | 0.0545        |
| KGSF (Zhou et al., 2020)                                                                                                | 0.0579        | 0.0921        | 0.1246        | 0.1433        | 0.0405        | 0.0503        | 0.0599        | 0.0662        |
| UniCRS (Wang et al., 2022)                                                                                              | 0.0722        | 0.1053        | 0.1344        | 0.1494        | 0.0548        | 0.0640        | 0.0726        | 0.0778        |
| NBCRS (Xie et al., 2024)                                                                                                | <u>0.0801</u> | <u>0.1290</u> | <u>0.1655</u> | <u>0.2019</u> | <u>0.0661</u> | <u>0.0833</u> | <u>0.0954</u> | <u>0.1048</u> |
| <b>LLM Prompting-Based Methods</b>                                                                                      |               |               |               |               |               |               |               |               |
| GPT-4o-mini (zero-shot)                                                                                                 | 0.0949        | 0.1348        | 0.1600        | 0.1687        | 0.0747        | 0.0877        | 0.0950        | 0.0973        |
| GPT-4o (zero-shot) (He et al., 2023)                                                                                    | 0.1106        | 0.1625        | 0.1992        | 0.2147        | 0.0861        | 0.1028        | 0.1136        | 0.1197        |
| CRAE (Zhu et al. (2025))                                                                                                | <b>0.1146</b> | <b>0.1715</b> | <b>0.2030</b> | <u>0.2212</u> | <b>0.0885</b> | <b>0.1065</b> | <b>0.1164</b> | <b>0.1227</b> |
| <b>LLM Post-Training-Based Methods (off-policy, <math>\mu = 2</math>)</b>                                               |               |               |               |               |               |               |               |               |
| Most results are <b>significant</b> as standard error are between 0.0020-0.0035 for R@5-20 and 0.0015-0.0025 for N@5-20 |               |               |               |               |               |               |               |               |
| Qwen2.5-0.5B (zero-shot)                                                                                                | 0.0021        | 0.0021        | 0.0021        | 0.0021        | 0.0023        | 0.0022        | 0.0021        | 0.0021        |
| + SFT                                                                                                                   | 0.0642        | 0.1027        | 0.1212        | 0.1308        | 0.0502        | 0.0625        | 0.0678        | 0.0704        |
| + SFT+Vanilla GRPO                                                                                                      | 0.0781        | 0.1176        | 0.1429        | 0.1538        | 0.0623        | 0.0748        | 0.0823        | 0.0852        |
| + SFT+ GSPO                                                                                                             | 0.0816        | 0.1243        | 0.1511        | 0.1641        | 0.0647        | 0.0784        | 0.0862        | 0.0896        |
| + SFT+Rank-GRPO (log)                                                                                                   | 0.0818        | 0.1256        | 0.1604        | 0.1764        | 0.0644        | 0.0784        | 0.0886        | 0.0930        |
| + SFT+Rank-GRPO ( $\exp_{\infty}$ )                                                                                     | <u>0.0935</u> | <u>0.1412</u> | <u>0.1738</u> | <u>0.1899</u> | <u>0.0735</u> | <u>0.0887</u> | <u>0.0982</u> | <u>0.1025</u> |
| Llama-3.2-1B (zero-shot)                                                                                                | 0.0187        | 0.0240        | 0.0257        | 0.0263        | 0.0157        | 0.0170        | 0.0175        | 0.0176        |
| + SFT                                                                                                                   | 0.0754        | 0.1148        | 0.1354        | 0.1498        | 0.0595        | 0.0723        | 0.0782        | 0.0819        |
| + SFT+Vanilla GRPO                                                                                                      | 0.0937        | 0.1415        | 0.1726        | 0.1974        | 0.0728        | 0.0882        | 0.0975        | 0.1042        |
| + SFT+GSPO                                                                                                              | 0.0901        | 0.1389        | 0.1693        | 0.1899        | 0.0700        | 0.0857        | 0.0947        | 0.1002        |
| + SFT+Rank-GRPO (log)                                                                                                   | 0.0937        | 0.1464        | 0.1832        | 0.2066        | 0.0731        | 0.0902        | 0.1011        | 0.1074        |
| + SFT+Rank-GRPO ( $\exp_{\infty}$ )                                                                                     | <u>0.1037</u> | <u>0.1555</u> | <u>0.1927</u> | <u>0.2166</u> | <u>0.0806</u> | <u>0.0973</u> | <u>0.1081</u> | <u>0.1146</u> |
| Llama-3.2-3B (zero-shot)                                                                                                | 0.0574        | 0.0805        | 0.0912        | 0.0961        | 0.0463        | 0.0535        | 0.0566        | 0.0580        |
| + SFT                                                                                                                   | 0.0788        | 0.1191        | 0.1410        | 0.1541        | 0.0615        | 0.0744        | 0.0807        | 0.0842        |
| + SFT+Vanilla GRPO                                                                                                      | 0.0952        | 0.1425        | 0.1726        | 0.1952        | 0.0766        | 0.0916        | 0.1004        | 0.1065        |
| + SFT+GSPO                                                                                                              | 0.0942        | 0.1436        | 0.1770        | 0.1943        | 0.0758        | 0.0914        | 0.1011        | 0.1058        |
| + SFT+Rank-GRPO (log)                                                                                                   | 0.1037        | 0.1572        | 0.1958        | 0.2220        | 0.0816        | 0.0989        | 0.1101        | 0.1169        |
| + SFT+Rank-GRPO ( $\exp_{\infty}$ )                                                                                     | <u>0.1092</u> | <u>0.1653</u> | <u>0.2010</u> | <b>0.2272</b> | <u>0.0845</u> | <u>0.1028</u> | <u>0.1132</u> | <u>0.1203</u> |

# Anexo: Formula Adjust

$$\mathcal{L}_{\text{adjust}} = - \sum_{i \in \mathcal{D}_{\text{train}}} \sum_{j: \mathbf{y}_{i,j}^{\text{gt}}=1} \log \frac{\exp((\mathbf{w} \odot \mathbf{s}_{\text{reflect},i} + \mathbf{b})_j)}{\sum_{k \in \mathcal{C}} \exp((\mathbf{w} \odot \mathbf{s}_{\text{reflect},i} + \mathbf{b})_k)} + \lambda_w \|\mathbf{w} - \mathbf{1}\|_2^2 + \lambda_b \|\mathbf{b}\|_2^2,$$

- : El negativo adelante convierte la maximización de verosimilitud en un problema de minimización (convención estándar en optimización).

$\Sigma_{i \in \mathcal{D}_{\text{train}}}$  : Suma sobre todas las conversaciones del dataset de entrenamiento.

$\Sigma_{j: \mathbf{y}_{i,j}^{\text{gt}}=1}$  : Dentro de cada conversación  $i$ , suma solo sobre los ítems  $j$  que tienen feedback positivo real del usuario (los groundtruth). Si el usuario dijo que le gustaron 3 películas, se suman esos 3 términos.

$\exp(\mathbf{w} \odot \mathbf{s}_{\text{reflect},i} + \mathbf{b})_j$  : Score ajustado del ítem groundtruth  $j$ , pasado por exponencial. Es el numerador del softmax, representando "qué tan alto puntúa el ítem correcto".

$\Sigma_{k \in \mathcal{C}} \exp(\mathbf{w} \odot \mathbf{s}_{\text{reflect},i} + \mathbf{b})_k$  : Suma de scores exponenciados sobre **todo el catálogo**  $\mathcal{C}$ . Es el denominador del softmax, el factor de normalización.

$\log(\text{numerador/denominador})$  : Log-probabilidad de que el ítem correcto  $j$  sea seleccionado del catálogo completo. Maximizar esto empuja los scores de ítems relevantes por encima del resto.

$\lambda_w \|\mathbf{w} - \mathbf{1}\|_2^2$  : Regularización L2 sobre  $\mathbf{w}$ , penalizando desviaciones desde 1. Esto fuerza que  $\mathbf{w}$  sea cercano a 1, manteniendo el ajuste multiplicativo pequeño (residual).

$\lambda_b \|\mathbf{b}\|_2^2$  : Regularización L2 sobre  $\mathbf{b}$ , penalizando desviaciones desde 0. Mismo principio: el sesgo aditivo debe ser mínimo.

# Anexo: Generación de metadata

## Prompt to Generate Metadata for Items Known to the Teacher LLM $\Theta$

Summarize the metadata for the movie `movie_name` (release year) based on the information in the format of:

**Movie\_Name (Release\_Year)**

**Keywords:**

<comma-separated keywords>

**Plots:**

<one-paragraph plot summary>

You should separate the keywords with commas without numbering.  
The plot should be one paragraph.

## Prompt to Generate Metadata for Items Unknown to the Teacher LLM $\Theta$ with RAG and ICL

Summarize the metadata for the movie `movie_name` (release year) based on the information in the format of:

**Movie\_Name (Release\_Year)**

**Keywords:**

<comma-separated keywords>

**Plots:**

<one-paragraph plot summary>

You should separate the keywords with commas without numbering.  
The plot should be one paragraph.

Here are the information related to the target movie: {movie\_info}

Here are some examples for the generated metadata for different movies: {examples}

(i) **Ítems del catálogo que el LLM profesor conoce:** Se usa un prompt zero-shot para generar un resumen con keywords y descripción de la trama.

(ii) **Ítems fuera del catálogo (OOC) o alucinados:** Como estos ítems fueron recomendados por el propio LLM, él aún puede generar metadata plausible sobre ellos.

(iii) **Ítems del catálogo que el LLM no conoce** (por ejemplo, ítems de nicho o muy recientes): Se usa RAG (generación aumentada con recuperación) e ICL (aprendizaje en contexto). Se busca información externa sobre el ítem vía Google Search API en Wikipedia, IMDb y Rotten Tomatoes, y se le entrega al LLM junto con ejemplos few-shot para guiar la generación de metadata.